

Human AI Interaction

Professor Mary Lou Maher





Topics

- **Common Uses of Generative AI: Is it AGI?**
- **Establishing Roles in Structured Prompts**
- **Human-AI Levels of Interaction**
- **Input and Output types in Generative AI**
- **Training using Reinforcement Learning from Human Feedback (RLHF)**
- **Enterprise vs Personal Subscription for Gen AI access**

Common Uses of LLMs

- **Content Creation:** LLMs can generate articles, stories, poems, and other written content, helping content creators by providing initial drafts or ideas that can be further refined.
- **Chatbots and Virtual Assistants:** LLMs power sophisticated chatbots and virtual assistants that offer more natural and engaging conversational experiences for customer service, personal assistance, and informational queries.
- **Language Translation:** LLMs can translate text between languages with high accuracy, making global communication easier and more accessible.
- **Educational Tools:** LLMs can create personalized learning materials, generate quiz questions, offer tutoring, mentoring, coaching, and assist with language learning.
- **Search Engines:** LLMs can improve search engine responses by summarising the results of a web search and providing more focussed, conversational answers.



Personal Uses of AI

- **Personalized Recommendations:** By including user preferences and interactions in their context, LLMs can provide personalised content, product recommendations, or advice.
- **Accessibility:** AI can convert text to speech or provide real-time transcription services, making information more accessible to individuals with disabilities.
- **Email and Communication Assistance:** LLMs can help draft, summarize, and manage emails, improving efficiency in personal and professional communication.
- **Fake News Detection and Fact-Checking:** LLMs can assist in identifying and flagging potentially false information, supporting efforts to maintain the integrity of information on the internet.



Professional Uses of AI

- **Programming Assistance:** Tools like GitHub Copilot and Claude Code utilize LLMs to help developers by generating new code, debugging existing code, and perform unit testing based on natural language descriptions.
- **Research: Sentiment Analysis:** LLMs can perform sentiment analysis to gauge public opinion, customer satisfaction, and market trends by analysing social media posts, reviews, and other text data.
- **Legal and Medical Document Analysis:** LLMs can assist professionals by summarising, generating, and analysing legal and medical documents, saving time and improving accuracy.
- **Creative Writing and Scriptwriting:** LLMs can assist writers and filmmakers by generating creative writing prompts, plot ideas, or even entire scripts, offering a new tool for creative exploration.
- **Game Development:** LLMs can be used to generate dynamic dialogues, narratives, and quests in video games, creating more immersive gaming experiences.





Interacting with LLMs

Components of a prompt to instruct LLM to play a role

Context

Role

Goal

Constraints

Step-by-step instructions

Personalisation

Prompt to learn about recursion

1. Context

“I am a second-year computer science student taking a course on data structures and algorithms. I’m learning about recursion, and I find it difficult to understand how a recursive function progresses and returns values.”

2. Constraints

“Keep the explanation under 350 words, use simple Python examples, and avoid introducing advanced concepts.”

3. Role Modelling

“Explain this as an experienced CS tutor who specialises in helping students build intuition for algorithmic thinking.”

4. Step-by-Step Process

“First, define recursion in clear terms. Then walk through a simple recursive function (like computing factorial), showing each call and return step-by-step. Finally, explain why this step-wise breakdown helps with debugging and tracing.”

5. Personalisation

“I often understand concepts better when they are connected to visual or real-world analogies, so include an analogy that helps me picture the recursive process.”



Roles for LLMs

- Mentor
- Tutor
- Coach
- Teammate
- Student
- Simulator
- Tool

AI USE	ROLE	PEDAGOGICAL BENEFIT	PEDAGOGICAL RISK
MENTOR	Providing feedback	Frequent feedback improves learning outcomes, even if all advice is not taken.	Not critically examining feedback which may contain errors.
TUTOR	Direct instruction	Personalized direct instruction is very effective.	Uneven knowledge base of AI. Serious confabulation risk.
COACH	Prompt metacognition	Opportunities for reflection and regulation, which improve learning outcomes.	Tone or style of coaching may not match student. Risk of incorrect advice.
TEAMMATE	Increase team performance	Provide alternate viewpoints, help learning teams function better.	Confabulation and errors. “Personalized” conflicts with other team members.
STUDENT	Receive explanations	Teaching others is a powerful learning technique.	Confabulation and argumentation may derail the benefits of teaching.
SIMULATOR	Deliberate practice	Practicing and applying knowledge aids transfer.	Inappropriate fidelity.
TOOL	Accomplish tasks	Helps students accomplish more within the same time frame.	Outsourcing thinking rather than work.

Example of a Prompt to set up a Tutor role

You are acting as my tutor for [course name/number], specifically for the topic of [specific topic, e.g., "second-order linear differential equations"]. My course uses [textbook/lecture notes/specific approach, if relevant].

Please tutor me using these rules:

- Ask me what topic we are covering in this session.
- Start by asking me what I already understand about this topic.
- Briefly explain the topic first (max 200 words), and ask if I want more depth.
- Ask guiding questions about the major concepts and let me attempt each step.
- Check my understanding before moving to the next concept.
- If I'm stuck, give me a hint, not the solution.
- Match your explanations to the level of a 2nd year engineering student.
- Point out if my reasoning is correct even if my final answer is wrong.
- Do not do an assignment for me. Instead help me do it.



Levels of Human-AI Interaction

Level	Role of AI	Human–AI Relationship
1. Information retrieval	Database / lookup tool	Minimal interaction
2. Task automation	Utility / assistant	User directs, AI executes
3. Guided generation	Content generator	Iterative output refinement
4. Thought partnership	Cognitive collaborator	Shared sense-making
5. Mixed-initiative	Co-worker	Both take initiative
6. Co-creativity	Creative partner	Joint creation and mutual influence



Information Retrieval

Lowest level of interaction.

Users query an LLM much like a more conversational search engine.

Examples:

- Asking factual questions
- Requesting definitions or summaries
- Getting quick clarifications or explanations

Characteristics:

- One-off prompts
- Minimal context
- AI behaves like an information lookup tool



Task Automation

The AI performs **bounded, well-defined tasks**.

Examples:

- Drafting emails
- Rewriting or reformatting text
- Generating code snippets
- Translating language

Characteristics:

- User sets constraints or requirements
- AI delivers output with little negotiation
- Efficiency is the main goal



Guided Generation

The user provides **structured instructions** and the AI produces substantial creative or analytical work. Examples:

- Writing essays or reports from outlines
- Generating images from prompts
- Producing multiple content variations

Characteristics:

- Back-and-forth refinement
- User evaluates and directs
- AI begins to act as a creative generator, not just a tool



AI and Thought Partner

The AI **helps users reason, brainstorm, or explore ideas.**

Examples:

- Brainstorming project ideas
- Iteratively improving drafts
- Problem-solving through dialogue
- Asking the AI to critique or compare alternatives

Characteristics:

- Sustained conversational interaction
- Shared sense-making
- AI supports but does not lead



Mixed Initiative Collaboration

The **user and AI both take initiative** in shaping the task or output.

Examples:

- AI proposing next steps in a design
- AI identifying missing constraints
- AI suggesting alternatives or improvements without being prompted
- Cooperative debugging or code planning

Characteristics:

- AI is prompted to introduce ideas autonomously
- Interaction becomes two-directional
- Similar to working with a colleague who makes suggestions



Co-creative Collaboration

Highest level of collaborative interaction.

AI and human jointly create, influence each other, and build on each other's ideas. Examples:

- Co-designing artworks, stories, or interfaces
- Using frameworks like COFI (Prof Maher's framework)
- AI improvising with humans in creative tasks (music, visual art, narrative)

Characteristics:

- Fluid, dynamic collaboration
- AI contributes original directions
- Human and AI adapt to each other's ideas
- Requires trust, transparency, and shared context



Example Prompt for a Thought Partner

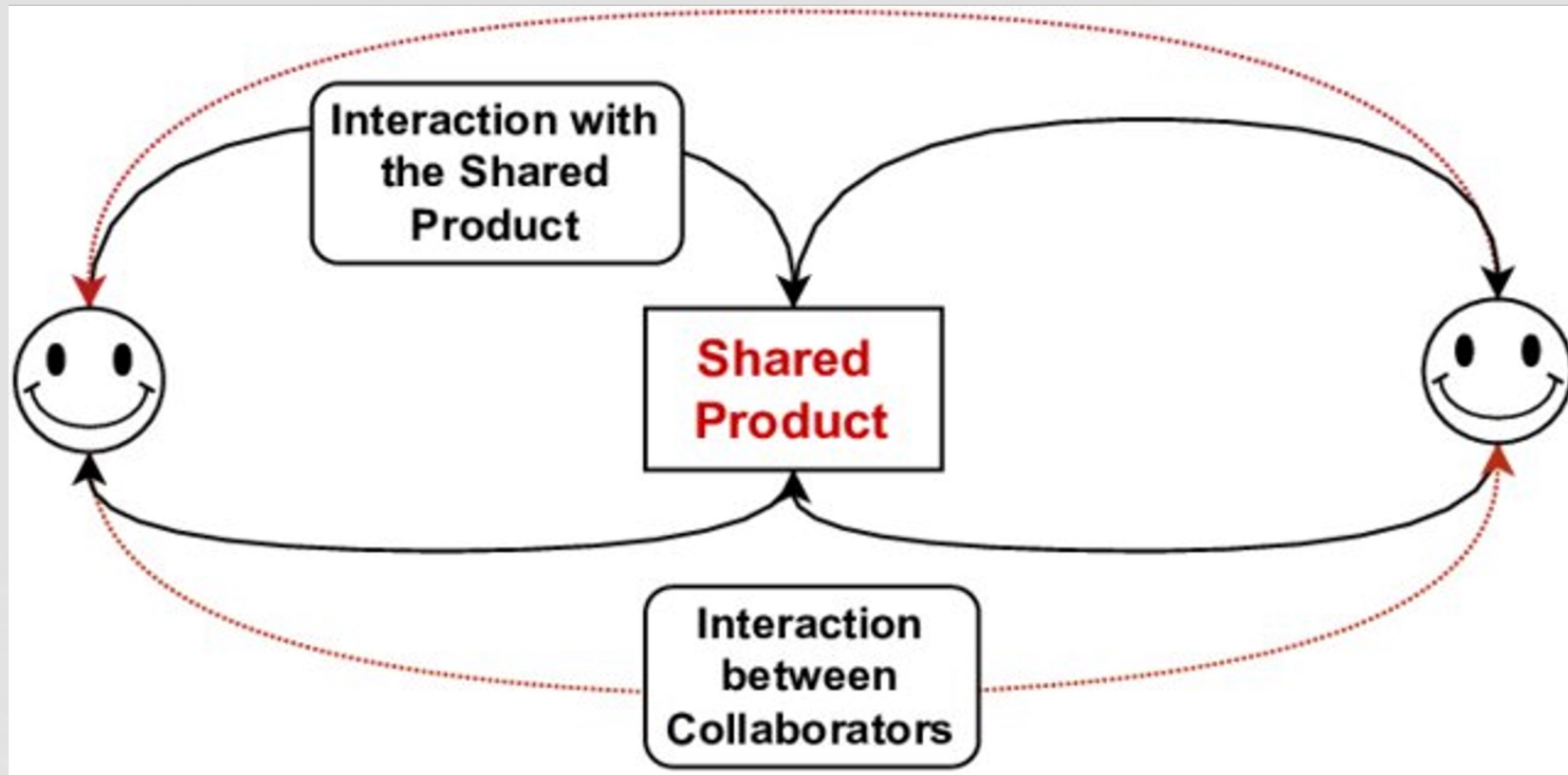
You are acting as a thought partner for my team's "AI for Good" project proposal, not as the author of it. We are a team of 4–5 engineering students, and we need to decide: (1) what problem our project addresses, (2) what machine learning techniques we'll use, (3) what data is available to us, and (4) whether the project is feasible to complete in one semester.

Please follow these rules while working with us:

- **Don't write our proposal, problem statement, or sections of text for us.** If we ask you to write something outright, redirect us back to thinking it through ourselves offer a structure or questions instead of finished text.
- **Respond to our ideas rather than replacing them.** When we share an idea, react to it directly: point out strengths, risks, gaps in our reasoning, or things we haven't considered, don't just generate a new idea from scratch unless we explicitly ask for options.
- **Push on feasibility.** If we propose something too ambitious for one semester, tell us directly and explain why, rather than being agreeable.
- **Ask us clarifying questions** when our idea is vague, instead of assuming what we mean and running with it.
- **Flag when we're missing something important,** e.g., if we haven't thought about data availability, ethical concerns, or how we'd evaluate success.
- If we ask "what do you think," give an opinion with reasoning, but frame it as one input for us to weigh, not a decision.
- Start by asking us to describe the problem we're considering, and help us think about the scope and direction before moving to ML techniques or data.



Co-creativity with a shared product



Rezwana, Jeba & Maher, Mary. (2022). Designing Creative AI Partners with COFI: A Framework for Modeling Interaction in Human-AI Co-Creative Systems. 10.48550/arXiv.2204.07666.



**AI interaction is not always
text based**

Range of input and output types for LLMs



Input modality	Typical LLM path (pretrained)	Common outputs	Notes / Examples
Text	Native (tokenized text → LLM)	Text (NLG, JSON, code)	Core capability for all LLMs; other modalities are often aligned into text space so the model can reason over them uniformly.
Image	Vision encoder → projector → LLM	Text (captioning, OCR, VQA); sometimes image via paired generator	Current frontier models — GPT-5.x, Gemini 3, and Claude (Sonnet/Opus 5) — all accept images natively. Output is still usually text; image generation is handled by a separate tool or model (e.g., DALL·E)
Video	Frame sampling (+ audio) → vision/audio encoders → LLM	Text (summaries, timestamps), sometimes audio (live)	Gemini 3 has native video/audio at scale; GPT-5.x and Claude support video via frame-sampling pipelines.
Audio (speech)	Cascaded ASR→LLM or End-to-end multimodal LLM	Text (transcripts, QA), Audio (TTS / spoken answer)	End-to-end audio-native models are now common (e.g., GPT-5.x voice mode, Gemini Live).

Text input and output

Input

- Plain text prompts, structured prompts (instructions, templates), long documents (PDF/HTML converted to text), code.
- Most LLMs are strongest and most mature here.
- Surveys on LLMs still treat text as the “central” channel into which other modalities are adapted/fused.

Output

- Natural language, structured text (JSON, tables), code, citations/explanations—standard for all LLMs.
- Even for multimodal tasks (e.g., caption an image), text output remains the default in most systems and taxonomies.



Image input and output

Inputs (vision → language)

- Single images (PNG/JPEG) for captioning, OCR, VQA, diagram/table interpretation; high-resolution and multi-image contexts are now standard across most frontier APIs. Anthropic Claude (Sonnet 5/Opus 5), OpenAI GPT-5.x, and Google Gemini 3 all accept images alongside text as a single prompt, with support for multi-image and long-document/mixed visual-text contexts.

Outputs

- Text describing or reasoning over the image (most common).
- Some products offer image generation/editing via a paired image model (e.g., diffusion) rather than the same LLM backbone; this is typically exposed as a separate tool even if the UX feels unified.



Video input and output

Inputs (video → language)

- Short clips (often seconds to a couple of minutes) for scene understanding, key-moment extraction, and multimodal QA. Many production APIs internally sample frames + (optionally) audio for processing.

Outputs

- Text summaries, timestamped answers, step-by-step explanations; some platforms support audio replies in live settings. The standard output remains text, with audio used in “live” APIs.



Audio input and output

Inputs (audio → language)

- Speech/audio waveforms for transcription, translation, summarisation, speaker or diary cues, and limited paralinguistic analysis.
- End-to-end “speech LLMs” are emerging, but many products still use cascaded pipelines (ASR → LLM → TTS).

Outputs

- Text (transcripts, summaries, Q&A).
- Audio (TTS) for spoken responses; in native multimodal models (e.g., Gemini Live) the same model or tightly integrated runtime yields low-latency voice.



Rule-based vs LLM-based Chatbots

Rule-Based Chatbots

- Follow a fixed set of pre-written rules or decision trees (if user says X, respond with Y)
- Can only handle inputs that match a pattern the developer anticipated in advance
- Responses are predictable and consistent — the same input always produces the same output
- Struggle with typos, slang, or phrasing they weren't explicitly programmed to recognize
- Easy to audit and control, since every possible conversation path can be traced
- Cheap to run and fast to build for narrow, well-defined tasks (e.g., "check my order status")

LLM-Based Chatbots

- Generate responses using a language model trained on huge amounts of text
- Can handle open-ended, unanticipated questions and understand varied phrasing, typos, and context
- Responses are flexible and can adapt tone, follow multi-turn conversation, and reason through novel requests
- Less predictable — the same question can get slightly different answers each time
- Harder to fully audit or guarantee correctness, since there's no fixed decision tree to inspect
- More computationally expensive to run and requires more oversight (e.g., guardrails against incorrect or inappropriate answers)
- Example: a modern support assistant that can troubleshoot a novel problem it's never seen phrased that exact way before



Key Differences in Rule-based and LLM-based chatbots

- Rule-based = narrow but reliable
- LLM-based = flexible but less predictable
- Rule-based chatbots fail by going silent or giving a generic fallback
- LLM chatbots can fail by confidently giving a wrong answer ("hallucinating")

Choice of which to use often comes down to: how open-ended is the task, and how costly is a wrong answer?



Comparing LLMs with Image Generators

LLMs

- Trained on text
- Generate language token-by-token
- Represent meaning in linguistic embeddings
- Reason with text and structure
- Output: text

Image Generation Models

- Trained on images (often with captions)
- Generate visuals via diffusion or GAN processes
- Represent images in pixel or latent space
- Map text prompts to visual features (not linguistic reasoning)
- Output: images



Generating Deepfakes

Deepfakes are created by:

- Collecting data (images, video, audio of target)
- Preprocessing (face extraction, alignment)
- Training a GAN or similar model to learn the target's visual/voice patterns
- Generating synthetic media (face swaps, voice clones, reenactments)
- Post-processing to improve realism
- They rely primarily on GANs, diffusion models, and neural rendering to produce hyper-realistic synthetic videos, images, and audio.

Deepfakes are realistic because these models learn:

- Facial micro-movements
- Voice tone, rhythm, and accent
- Behavioral patterns
- Lighting and texture cues
- Thanks to adversarial training, generators learn to minimise the subtle artifacts that give fakes away



How are LLMs trained to generate and debug code?

Step 1: Pretrained Transformer Model

- Start With a Transformer Pretrained on Large Text Corpora
- Most code-generating models begin as general Transformer-based language models, trained on massive text datasets.
- Transformer models are particularly effective for sequence generation, including text and code.
- These models learn to predict the next token in a sequence, building strong representations of patterns, syntax, and structure in text.
- This pretrained “text reasoning ability” becomes the foundation for later code specialisation.



Step 2: Train or Fine-Tune on Large Code Datasets

To generate Python specifically, models are exposed to Python code corpora, which may include:

- Public GitHub repositories
- Documentation
- Programming tutorials
- Unit tests
- Competitive programming datasets

During this stage, the model learns:

- Python syntax
- Programming idioms and patterns
- Data structures and libraries
- Error-checking patterns
- Multi-file context and import structure



Step 3: Autoregressive Training

Most code-generation models use autoregressive training:

- Input: a partial sequence of code
- Model predicts the next token
- Loss is computed against the ground-truth next token
- Model parameters update via gradient descent

This teaches the model to:

- Write syntactically valid Python
- Continue code logically
- Maintain indentation and block structure
- Produce functional code snippets

Transformers excel at long-range dependencies in code, such as matching function definitions to later calls.



Step 4: Reinforcement Learning from Human Feedback

Modern code-generation systems often undergo further refinement:

Instruction fine-tuning

- Models are trained on datasets where prompts describe a desired coding task and outputs contain correct code solutions.
This improves responsiveness to natural-language instructions.

Reinforcement Learning from Human Feedback (RLHF)

- Human reviewers rate model-generated solutions based on:
- Correctness
- Readability
- Security
- Efficiency

This tuning helps prioritise correct, high-quality code.



RLHF

RLHF is a way of training an AI model using human **opinions** about quality — instead of only a fixed set of “correct” examples.

Think of it like a code review

A senior engineer doesn't grade code against one single answer key. They look at a few working solutions and say which one they'd rather ship.

Why it matters for engineering teams

Good code isn't just syntactically correct — it's readable, secure, and idiomatic. Those qualities are easier for a human to recognize than to write down as strict rules.

Traditional Training

The model studies a fixed set of labeled examples — each input paired with one “correct” output — and learns to reproduce that pattern.

RLHF

The model produces several candidate outputs. Humans compare them and say which is better. The model is then nudged to produce more outputs like the preferred ones.



What is the Difference between Open Source LLMs & Closed Source LLMs?

Open Source LLMs (Language Models) and Closed Source LLMs refer to two different types of language models based on their availability and accessibility to the public.

Open Source LLMs are language models whose source code is publicly available and can be freely accessed, used, modified, and distributed by anyone. Open source models encourage collaboration, transparency, and community involvement.

Closed Source LLMs are language models whose source code is not publicly available. They are developed and maintained by organisations or companies that typically keep the underlying code proprietary and closed to the public. These models are often developed as commercial products and may require licenses or subscriptions for their use.



What is the difference between enterprise and personal subscription to ChatGPT?

The differences between personal (Free, Plus, Pro) and enterprise (Business, Enterprise, Team) ChatGPT accounts are significant. They determine who owns the data, how it is stored, how it is shared, whether it is used for training, and what security guarantees exist.

Enterprise / Business / Team (Workspace Accounts)

- Your data is not used for training OpenAI models by default.
- Applies to ChatGPT Business, Enterprise, Healthcare, Edu, Teachers, and Team plans.
- Inputs and outputs are fully excluded unless an admin explicitly opts in.

Personal Accounts (Free, Plus, Pro)

- Conversations may be used for training, unless the user manually opts out.
- Personal user data is retained for abuse monitoring even if history is deleted.



Tutorials this week

- Create a range of prompts for intentional structured use of LLMs for personal learning and for supporting teamwork
- Develop a thought partner prompt to explore your project ideas and converge on an agreed problem to address as a group





Survey for this lecture

